

Doxa and Logos: An Operator Model of the Interaction between a Language Model and a Formal System

Vladimir Lapshin*

Abstract

Large language models translate text into a formal language well, but they cannot reason soundly: they do not distinguish a correct step from a plausible one, they do not track what an answer rests on, and they do not refuse to guess. Symbolic solvers can do all of this, but they cannot read text. We propose to treat this division not as an engineering compromise but as the epistemological distinction between *doxa* and *logos*: *doxa* is a source of proposals that have no derivation inside the system; *logos* is a formal system that accepts or rejects them by strict rules. We introduce a family of interaction operators between *doxa* and *logos* and distinguish three roles within it: external (doxastic) operators supply a proposal not derivable from the theory; internal (logotic) operators compute the system's reaction; protocol operators record, classify, and control the exchange. The central claim is that this interface is asymmetric. We show that the distinction has rigorous formal analogues — doxastic logic, non-monotonic logics, abduction — and that each of them describes one of the interaction operations. We also describe a working system, Ankyra, in which a large language model plays the role of *doxa* and a symbolic engine plays the role of *logos*.

1 Introduction

Formal systems have a fundamental limit. Gödel's incompleteness theorems showed that any sufficiently rich consistent system contains statements it can neither prove nor disprove [1]. Turing's halting problem showed that no general algorithm exists that, for any program and input, would determine whether it halts [2]. These results are usually interpreted as absolute limits of cognition.

We propose a different interpretation. These are the limits of *static* systems — closed

*merfill@yandex.ru

sets of axioms and inference rules. But human thinking is not a static system. It does not begin with a fixed set of axioms and derive consequences to exhaustion. It constantly expands, revises, and augments its formalisms, relying on what the ancient tradition called *doxa*.

The distinction between *doxa* and *logos* has a long history. Already in Xenophanes (6th c. BCE) *doxa* ($\delta\acute{o}\xi\alpha$) denotes *opinion or belief*, fundamentally opposed to certain knowledge (truth). Parmenides (5th c. BCE) contrasts $\delta\acute{o}\xi\alpha$ with *truth* ($\alpha\lambda\eta\theta\epsilon\iota\alpha$): *doxa* is the “deceptive order” of the sensory world, whereas truth is accessible only to *logos* — pure thought that does not trust the evidence of eyes and ears [3]. It is in Parmenides that the distinction between the data of the senses and the data of *logos* becomes radical: only the latter, on his view, is real and true.

Plato radicalized this distinction. For him, $\delta\acute{o}\xi\alpha$ is cognition of *what appears*, whereas *episteme* ($\epsilon\pi\iota\sigma\tau\acute{\eta}\mu\eta$) is cognition of *what is*. *Doxa* may be true or false, but it always remains *unstable, finite, contingent* — it “is expressed by relation to an object, not by justification of truth” [4]. *Episteme*, by contrast, is knowledge *accompanied by logos*, that is, by explanation, justification, *rigorous reasoning*.

In the *Theaetetus*, Plato discusses and ultimately rejects the definition of knowledge as “true opinion with logos.” This matters for us: *logos* is not simply “added” to *doxa*; it works differently.

Aristotle then refined this: *doxa* is *opinion*, which may be true but lacks *necessary* character; *episteme* is knowledge of *the necessary*, obtained through *demonstration* [5].

Thus, in the ancient tradition a stable distinction emerged: *doxa* is opinion, experience, probabilistic judgment rooted in the sensory and cultural; *logos* (or *episteme*) is rigorous, justified, verifiable knowledge. *Doxa* is *wider*, but *less reliable*. *Logos* is *narrower*, but *guaranteed*.

Before identifying these terms with anything in a system, let us fix what distinguishes *doxa* from *logos*. Not through negation (“*doxa* is not formalized knowledge”), but through positive characteristics.

- **Historicity.** Experience accumulates in layers: the new does not cancel the old but lies on top of it. *Logos* expands by adding axioms, but it does not layer.
- **Associativity.** *Doxa* links the heterogeneous — it sees similarity where formally there is none. *Logos* links only what is linked by inference rules.
- **Immediacy.** *Doxa* gives an answer without derivation, before verification. *Logos* reaches an answer only through derivation.
- **Contextuality.** In *doxa*, meaning is determined by context. In *logos*, meaning is

fixed by the language L .

- **Probabilism.** Doxa gives a degree of confidence, not a binary. Logos gives “proven” or “not proven”, with nothing in between.

These are characteristics of a type of knowledge, not of its bearer. They do not claim anything about how doxa is “really” arranged; they fix how it *differs* from a formal system when it is regarded as a source of proposals.

In this paper we use a large language model in the role of doxa. This is not a metaphor and not an analogy: the LLM satisfies all five characteristics.

- **Historicity.** The LLM is trained on accumulated human text, and its weights carry traces of many epochs and genres. Fine-tuning adds layers without erasing previous ones.
- **Associativity.** Attention links tokens by statistical proximity, not by inference rules. The model sees similarity where formally there is none.
- **Immediacy.** The answer is produced in one direct pass, without explicit derivation and without intermediate verification.
- **Contextuality.** The meaning of a token depends on the entire context window; the same word in different contexts receives a different representation.
- **Probabilism.** The output of the model is a probability distribution over the next token, not a binary verdict.

It is precisely this correspondence that makes the identification meaningful. If the LLM did not possess these characteristics, using it in the role of doxa would be impossible, and the distinction would remain a philosophical metaphor. It is not one.

Why such an identification becomes possible only now. The formal side has been developed for centuries: logic, proof theory, decision procedures, specification languages. Logos was and remains the subject of exact research. Doxa, by contrast, remained without an instrument: it was described philosophically, but could not be addressed by a machine. The large language model is the first instrument of this kind. That is why a distinction known for two and a half thousand years becomes an *architectural principle* only now, rather than a philosophical metaphor.

Logos in our system is a symbolic solver: a formal system that accepts or rejects the proposals of doxa by strict rules. The distinction between doxa and logos does not mean that one cancels the other. Human thinking relies on both sources: doxa supplies content, logos verifies and holds it. In this paper we are interested not in the nature of doxa as such, but in the *interface* between it and a formal system.

The key thesis of the paper is the following. Although doxa is not a theory in the language of the formal system, *the functions of its interaction with that system* can be described formally. We introduce a family of interaction operators and distinguish three roles within it: *external* (doxastic) operators supply a proposal not derivable from the theory; *internal* (logotic) operators compute the system’s reaction to the proposal; *protocol* operators record, classify, and control the exchange. The fate of the proposal is decided by logos — it *does not produce* it itself.

The paper is structured as follows. In Section 2 we examine the theoretical foundations: the distinction between doxa and logos in the ancient tradition and in modern logic, a survey of doxastic logic, non-monotonic logics, abduction, and dynamic logics of belief. In Section 3 we introduce the family of interaction operators between doxa and logos and its formal properties. In Section 4 we describe a concrete implementation of this model. In Section 5 we discuss the limits of the approach, its relation to hypercomputation, and practical implications for explainable AI. Section 6 concludes the paper.

2 Theoretical foundations

In Section 1 we introduced the distinction between doxa and logos and showed its ancient origins. Here we will not repeat the history. Our task is to show that this distinction has *rigorous formal analogues* in modern logic and that these analogues are *relevant* to our model.

We consider three approaches: doxastic logic, non-monotonic logics, and abduction. Each captures a separate aspect of the interaction between doxa and logos. None of them describes doxa as such — and this is not required: we are interested not in the nature of doxa, but in those functions through which it extends a formal system. These functions are what we describe in Section 3.

2.1 Doxastic logic

The term “doxastic logic” was introduced by *Hintikka* in 1962 for the formal modeling of an agent’s *opinions and beliefs* (δόξα), as distinct from knowledge (ἐπιστήμη) [6]. The basic operator B_α reads as “Alpha believes that...”. Doxastic logic deals with *synchronic* principles — how beliefs are related to one another at a given moment.

But it is *silent* about how beliefs change over time. This limitation was recognized, and the response to it was *dynamic doxastic logic* (DDL), introduced by *Krister Segerberg* in the 1990s [7]. DDL adds to static doxastic logic *modal operators of belief change*: $[*p]q$ reads as “ q holds after revision of beliefs by p ” [8].

Thus DDL is a formal analogue not of doxa as such, but of the *operator of revision of the*

formal system when a new proposal arrives from *doxa*. This is precisely what we call a *function of doxa*.

2.2 Non-monotonic logics

Classical logic is *monotonic*: if $\Gamma \vdash \phi$, then $\Gamma \cup \Delta \vdash \phi$. Adding new axioms *does not cancel* previously derived consequences. But a formal system interacting with *doxa* works differently. New experience may *cancel* old conclusions. If I know that Tweety is a bird, I assume he can fly. If I later learn that Tweety is a penguin, I *cancel* the conclusion “Tweety flies,” although it previously followed from “Tweety is a bird.”

Non-monotonic logics are formal systems that model precisely this *defeasibility* of conclusions. Classical approaches include:

- **Default Logic** by *Reiter*: rules of the form “if A and there is no information about exceptions, then B ,” which can be defeated when an exception appears [9].
- **Circumscription** by *McCarthy*: minimization of “abnormal” situations, allowing default conclusions [10].
- **Autoepistemic logic** by *Moore*: modeling an agent’s beliefs about its own beliefs [11].

All these formalisms acknowledge that *the conclusions of a formal system are not final*. They are *defeasible* when new information from *doxa* arrives. And this is not a “deficiency.” It is a *property* of a system that is *open* to interaction with *doxa*. Non-monotonic logics formalize this property but *do not eliminate* it. They show how a system can *work* with defeasible conclusions without losing coherence.

2.3 Abduction

If non-monotonic logics describe the *cancellation* of conclusions, then *abduction* describes the *generation* of new hypotheses. The term was introduced by *Charles Sanders Peirce* (1839–1914), who distinguished three types of inference: deduction, induction, and *abduction* — “the creative generation of explanatory hypotheses” [12].

Peirce characterized abduction as reasoning “from consequence to cause.” If we observe a fact Q , and know that $P \rightarrow Q$, abduction allows us to *suppose* P as a *possible explanation* of Q . This is not deduction (where P *necessarily* follows) nor induction (where P *probably* follows). It is the *generation of a hypothesis* that *explains* the observation [13].

In contemporary philosophy of science, abduction is often identified with “*inference to the best explanation*” (IBE). However, as researchers note, *the identification of abduction with IBE is not rigorous*. Peirce distinguished the *generation* of a hypothesis (abduction in

the narrow sense) from the *selection* of the best one (IBE). For our model, it is precisely *generation* that matters: doxa *proposes* hypotheses, and the formal system *verifies* them [14].

Abduction is *formalizable* — but only on condition that we formalize the entire “pattern” of abductive inference, including the rules of hypothesis generation and justification. There exist formal systems in which abduction is added to propositional logic as an *external extension operator* [15]. This means that abduction can be described not as an inference rule inside the logic, but as an operation over the theory that supplies a new proposal to it.

2.4 The JSM method: a synthesis of cognitive procedures

Abduction describes the *generation* of a hypothesis. A separate question is how a hypothesis is *accepted* as a working one if it does not follow from the theory. A formal approach to this question was proposed by *V. K. Finn* within the framework of the *JSM method for automatic hypothesis generation* [29]. The method is named after John Stuart Mill and formalizes his inductive methods by means of modern logic.

The key idea of the JSM method is that a cognitive procedure is not reducible to induction, analogy, or abduction taken separately. It is a *synthesis* of the three: induction generates hypotheses about causal relations on the basis of similarity and difference of objects; analogy transfers these hypotheses to new objects; abduction checks the condition of causal completeness — whether the causes found suffice to explain the observed effect [30, 31].

For our model, the JSM method matters for two reasons. First, it shows that *generation* and *acceptance* of a hypothesis are different operations, and the second does not reduce to the first. Second, a hypothesis in the JSM method receives a *degree of plausibility*, not a binary status “true/false,” which corresponds to the probabilistic nature of doxa (Section 1). The JSM method can be regarded as yet another formal analogue of an *external* operator: it describes how a hypothesis enters the theory with an explicitly stated ground.

2.5 Limits and complexity

The four approaches describe different operations over a formal theory. Doxastic logic and DDL work with an agent’s beliefs as a formalized slice; non-monotonic logics describe the cancellation of conclusions when new information arrives; abduction supplies a new proposal to the theory; the JSM method generates hypotheses and assesses their plausibility through a synthesis of induction, analogy, and abduction. None of them works with

doxa as an object: all four deal with operations, not with the source. It is precisely these operations that we call *functions of doxa*.

A separate difficulty is associated with these operations. Non-monotonic logics are computationally more complex than classical logic [16], and for many of them the entailment problem is undecidable or of very high complexity [17]: no general algorithm exists that, for any theory and goal, would determine whether the goal is a consequence. A procedure can be built, but its termination cannot be guaranteed. Abduction generates a multitude of hypotheses, and choosing among them is a separate task without a formal criterion of truth: propositional abduction is Σ_2^P -complete [18], and logical abduction is computationally hard in the general case [19]. The JSM method, being a synthesis of three cognitive procedures, inherits their complexity: checking causal completeness and assessing the plausibility of hypotheses require enumeration of subsets of features and in the general case do not reduce to polynomial procedures [29].

2.6 Connection to our model

The functions of doxa — how it extends a formal system — we describe formally. Each of the four approaches above corresponds to one or more operators, and each operator belongs to one of three roles:

- **Doxastic logic** $\rightarrow$ the *fixation* operator: what the agent “holds” as beliefs. This is a *protocol* function of the environment, implemented by the journal.
- **Dynamic doxastic logic** $\rightarrow$ the *revision of the formal system* operator. This is an *internal* operator: *logos* computes how the theory changes when new information arrives from doxa.
- **Non-monotonic logics** $\rightarrow$ the *cancellation* operator. This is an *internal* operator: *logos* recomputes what follows when exceptions obtained from doxa appear.
- **Abduction and the JSM method** $\rightarrow$ the *hypothesis generation* operator. This is an *external* operator: doxa supplies a hypothesis that the formal system verifies. Abduction describes the generation itself; the JSM method adds to it an assessment of plausibility through a synthesis of induction, analogy, and abduction.

This is *not* an exhaustive list. We will return to it in Section 3, where we introduce the *family of interaction operators between doxa and logos* and formal definitions of the roles. It is already clear: the distinction between doxa and logos rests on formal analogues in modern logic, and these analogues can be used to build a working system.

3 A family of interaction operators between doxa and logos

In Section 2 we showed that the distinction between doxa and logos rests on formal analogues in modern logic. Here we introduce the *family of operators* through which doxa and logos interact. We are interested not in the content of doxa, but in the *protocol* of its use.

The section is structured as follows. First we give basic definitions and the criterion by which operators are divided into three classes (Section 3.1). Then we describe each class separately: external operators, internal operators, and protocol operators. After that we introduce the role of the Arbitrator — an external instance that belongs neither to doxa nor to logos. Finally, we formulate properties of the family, describe the process of reasoning as a sequence of acts of consulting doxa, and clarify the role of logos in this process.

The central claim of the section is that this protocol is *asymmetric*: doxa supplies the content of proposals, whereas everything that decides, records, and controls their fate belongs to logos and the interaction environment.

3.1 Basic definitions and the role criterion

Let $F = (A, R, L)$ be a formal system with axioms A , inference rules R , and language L , and let $\text{Cn}(F)$ be the set of its consequences. Let D be doxa (Section 1): a source of proposals for F , which is not part of it and does not form a theory in L .

An operator may extend the *theory* (new axioms, rules, or objects) or the *language* itself; changing L changes what counts as a formula, and the two kinds of extension must not be confused. In what follows we consider only extensions of the theory: the language L is assumed fixed.

Let $\Sigma = \langle F, \mathcal{J}, \Pi \rangle$ be the state of the system, where F is the theory, $\mathcal{J}$ is the journal, and Π is a sequence of proposed but not yet accepted proposals. An operator is a function $o : \Sigma \times X \rightarrow \Sigma$, where X is the input (an observation, new information, a context, or an empty input). Classification is by what exactly the operator changes in Σ : the theory, the proposal queue, or the journal.

- **External** (doxastic) operator *produces a proposal*: content whose source is doxa. Formally: $o(\langle F, \mathcal{J}, \Pi \rangle, x) = \langle F, \mathcal{J}, \Pi \frown \langle P \rangle \rangle$. The theory and the journal do not change; the proposal enters Π and awaits a decision by logos.
- **Internal** (logotic) operator *produces a new theory*: F' , computed from F and the input. Formally: $o(\langle F, \mathcal{J}, \Pi \rangle, x) = \langle F', \mathcal{J}, \Pi \rangle$, where F' is a deterministic function

of F and x under fixed rules. Neither the journal nor the queue Π is changed by the operator: extracting the input from Π and recording the act is the work of the protocol operator (Section 3.4).

- **Protocol** operator *extracts a proposal and produces a record*: it does not change F . Formally: $o(\langle F, \mathcal{J}, \Pi \rangle, x) = \langle F, \mathcal{J}', \Pi' \rangle$, where $\mathcal{J}' = \mathcal{J} \cup \{\text{record of the act}\}$, and Π' is the queue Π without the extracted proposal x .

Class	Composition	What it changes in Σ
External (doxastic)	$o_{\text{abd}}, o_{\text{anal}}, o_{\text{int}}$	add a proposal to Π ; F and $\mathcal{J}$ do not change
Internal (logotic)	$o_{\text{rev}}, o_{\text{def}}, o_{\text{spec}}, o_{\text{ctx}}$	compute a new theory F' from F and the input; $\mathcal{J}$ and Π do not change
Protocol (environment)	o_{fix}	supplement the journal $\mathcal{J}$; F and Π do not change

3.2 External operators (doxastic)

External operators change neither the theory F nor the journal $\mathcal{J}$; they add a proposal to the queue Π , whence logos later retrieves it. The record of the act is made by the protocol operator (Section 3.4).

Hypothesis introduction operator (o_{abd}). Takes the state $\langle F, \mathcal{J}, \Pi \rangle$ and a pair $\langle Q, P \rangle$ proposed by doxa — an observation Q and a hypothesis P explaining Q — and appends P to the end of Π :

$$o_{\text{abd}}(\langle F, \mathcal{J}, \Pi \rangle, \langle Q, P \rangle) = \langle F, \mathcal{J}, \Pi \frown \langle P \rangle \rangle,$$

where four conditions hold: $P \notin \text{Cn}(F)$ (the hypothesis is new), $Q \notin \text{Cn}(F)$ (the observation is not derivable from the theory and therefore requires explanation), $Q \in \text{Cn}(F \cup \{P\})$ (the hypothesis explains the observation), and $F \cup \{P\}$ is consistent.

The consistency condition is essential: if $F \cup \{P\}$ is inconsistent, then by the principle of explosion any Q follows from it, and the explanation condition becomes trivial. The condition $Q \notin \text{Cn}(F)$ is equally essential from the other side: if the observation is already derivable from F , the abductive introduction of P explains nothing.

The explanation condition by itself does not require *minimality* of the hypothesis: if P explains Q , then $P \wedge R$ also explains Q . The selection of hypotheses without redundant conjuncts is performed at the stage of protocol classification (Section 3.10).

The theory does not change; the proposal enters Π and awaits a decision by logos. This is the formal analogue of abduction: the hypothesis is *proposed*, not derived [15].

Analogy operator (o_{anal}). Takes the state $\langle F, \mathcal{J}, \Pi \rangle$ and a pair of domains $\langle F_s, F_t \rangle$ — a familiar domain F_s and a new domain F_t — and adds to Π a proposal P_{analog} transferring the structure of F_s to F_t :

$$o_{\text{anal}}(\langle F, \mathcal{J}, \Pi \rangle, \langle F_s, F_t \rangle) = \langle F, \mathcal{J}, \Pi \frown \langle P_{\text{analog}} \rangle \rangle,$$

where $P_{\text{analog}} \notin \text{Cn}(F)$. The choice of the structural mapping is external; logos only checks whether it is preserved.

Intuition operator (o_{int}). Takes the state $\langle F, \mathcal{J}, \Pi \rangle$ and a goal ϕ and adds to Π an answer ϕ' that does not follow from F :

$$o_{\text{int}}(\langle F, \mathcal{J}, \Pi \rangle, \phi) = \langle F, \mathcal{J}, \Pi \frown \langle \phi' \rangle \rangle,$$

where $\phi' \notin \text{Cn}(F)$. This operator is closest to an oracle. Doxa *gives* the answer; logos verifies it if it can, otherwise defers it as a hypothesis. What is oracle-like here is the *source* of the answer, not the bearer, which may well be computable.

3.3 Internal operators (logotic)

These operators are often mistaken for functions of doxa because they are triggered by its input. In fact it is logos that computes them; the input is only their argument. Internal operators change the theory F but do not touch the journal $\mathcal{J}$: the record of the act is made by the protocol operator (Section 3.4).

The input of an internal operator comes from one of two sources. The first is a *proposal extracted from the queue* Π by a protocol operator: the protocol removes it from Π , records the act in the journal, and passes the proposal to the internal operator as an argument. This happens in revision and cancellation. The second source is an input already present in F (conflicting rules in specificity selection) or *declared by the protocol* (context); in this case Π does not participate. In all cases the internal operator itself changes only F ; changing Π and $\mathcal{J}$ is the prerogative of the protocol.

Revision operator (o_{rev}). Given the state $\langle F, \mathcal{J}, \Pi \rangle$ and new information p arriving from the queue Π , returns F' in which beliefs are revised in light of p :

$$o_{\text{rev}}(\langle F, \mathcal{J}, \Pi \rangle, p) = \langle F', \mathcal{J}, \Pi \rangle,$$

where F' is a deterministic function of F and p under a fixed selection function implementing *belief base revision* in the spirit of AGM principles [20]. The selection function is considered *part of the formal system* F : it enters its structure alongside axioms and inference rules and is not a hidden parameter of the state Σ . Thus F here is a base of axioms

and rules together with a preference ordering, not a logically closed set of beliefs; hence the talk is of base revision. Doxa supplies p ; logos decides how p affects F . Extracting p from Π and recording the act is performed by the protocol operator (Section 3.4); the internal operator changes only F .

Cancellation operator (o_{def}). Given the state $\langle F, \mathcal{J}, \Pi \rangle$ and an exception e arriving from the queue Π , returns F' in which previously derived consequences are cancelled:

$$o_{\text{def}}(\langle F, \mathcal{J}, \Pi \rangle, e) = \langle F', \mathcal{J}, \Pi \rangle.$$

This is the formal analogue of non-monotonic logics. Cancellation does not mean deleting axioms: what was derivable ceases to be derivable under the new configuration. Extracting e from Π and recording the act is performed by the protocol operator.

Specificity operator (o_{spec}). Given the state $\langle F, \mathcal{J}, \Pi \rangle$ and two rules r_1, r_2 present in F , returns F' in which the more specific one is selected:

$$o_{\text{spec}}(\langle F, \mathcal{J}, \Pi \rangle, \{r_1, r_2\}) = \langle F', \mathcal{J}, \Pi \rangle.$$

Specificity is a relation *computed by logos* over the hierarchy of F ; doxa does not supply it. This is the formal analogue of specificity preference in non-monotonic logics. Since the rules are already in F , the queue Π does not change.

Context application operator (o_{ctx}). Given the state $\langle F, \mathcal{J}, \Pi \rangle$ and a declared context Γ , returns F' to which the axioms of the context Γ have been added:

$$o_{\text{ctx}}(\langle F, \mathcal{J}, \Pi \rangle, \Gamma) = \langle F \cup \Gamma, \mathcal{J}, \Pi \rangle.$$

The context is *declared* by the protocol; its *application* is internal. Since Γ is not a proposal from the queue, Π does not change.

3.4 Protocol operators (environment)

These functions belong neither to doxa nor to logos. They introduce no content and do not change the theory F ; they change only the journal $\mathcal{J}$ and thereby make the exchange well-defined, deterministic, and verifiable.

Strictly speaking, only fixation is an operator in the sense of Section 3.1: it has the form $o : \Sigma \times X \rightarrow \Sigma$. The other protocol functions — classification, declaration, grounding checks, the wave protocol — are rules and procedures specifying how exactly the record is made. The general form of the operator part is:

$$o_{\text{fix}}(\langle F, \mathcal{J}, \Pi \rangle, x) = \langle F, \mathcal{J}', \Pi \rangle, \quad \mathcal{J}' = \mathcal{J} \cup \{\text{record of the act}\}.$$

Journal and fixation (o_{fix}). Each step is recorded in the journal: operator identifier, wave, rationale, source, and result (accepted, rejected, deferred). The journal is the protocol of interaction, not part of either doxa or logos. Fixation means selecting the formalized slice of doxa that is being consulted, and recording this act; it is an accounting action of the environment, not a function of doxa.

Deterministic classification. Every proposal from Π is classified by fixed rules (Section 3.10), independently of “mood”; this is what makes the exchange reproducible.

Declaration of context and fragment. The context Γ or the required logic fragment is *declared* as a run parameter; its application is internal (see o_{ctx} above).

Grounding checks. Checks of verbatim quotations and other structural checks certify that a proposal is anchored in the source.

Wave protocol. The rule “at most one proposal per act of consultation” makes each act verifiable and leaves a trace of where and why the theory changed.

3.5 Acceptance of proposals

The set Π contains proposals awaiting a decision. The decision is made by logos through protocol classification (Section 3.10). Three outcomes are possible.

Accepted. The protocol extracts the proposal from Π and passes it to the corresponding internal operator. The latter applies the proposal to F ; if the proposal rests on a quotation from the source, it enters F as an axiom; if it is an assumption, it enters with the mark H and is taken into account in assessing the strength of the answer. The protocol records the act in the journal $\mathcal{J}$.

Rejected. The protocol extracts the proposal from Π , does not pass it to the internal operator, and records the refusal in the journal with a reason code. The theory F does not change.

Deferred. The proposal remains in Π until the next wave or until the budget is exhausted.

In all three cases the protocol is responsible for extracting the proposal from Π and for the record in $\mathcal{J}$, while the internal operator is responsible for changing F . The journal stores the history: which proposal was considered and what decision was made. Π itself stores no history: it contains only awaiting proposals.

3.6 The Arbitrator

So far we have described operators that supply, compute, and record. But a question remains: who decides when the process stops, which of the competing proposals of doxa

is accepted, and whether the goal has been reached? This question requires a separate role.

The Arbitrator is an external instance that belongs neither to doxa nor to logos, but for which the system exists. The Arbitrator produces no content and computes no reaction of logos; it *selects* which operator from the family to apply to the current state.

The role of the Arbitrator is modeled as an external, with respect to the system, selection function $W : \Sigma \rightarrow \mathcal{O}$, where Σ is the state of the system and $\mathcal{O}$ is the set of operators of the family. The type of the function is given, but its algorithmic implementation lies outside the formal model Σ — otherwise we would have infinite regress (the Arbitrator would itself be subject to verification, and a meta-Arbitrator would be required).

The Arbitrator may be:

- a *human* — a researcher, user, or expert who makes decisions where a formal criterion is absent or insufficient;
- an *external program* — a test runner, verifier, or oracle that returns an objective verdict (for example, whether a test passed);
- a *composition* — for example, an automatic arbitrator makes decisions in known cases, and a human is involved only when the automatic one abstains.

The model *deliberately does not automate* the Arbitrator’s step fully. No formal criterion resolves a dispute between incompatible proposals of doxa if both are compatible with the theory. The choice remains with the one who is responsible for it — a human or an external program acting by a fixed criterion.

3.7 Properties of the family

External operators do not commute, and the order of their application matters. Let us examine this with an example. Let the theory F contain `is_a(tweety, bird)` and the default `bird  $\Rightarrow$  fly`, and let doxa propose `is_a(tweety, penguin)`.

First order: first o_{abd} , then acceptance of the proposal, then o_{spec} . After o_{abd} the proposal `is_a(tweety, penguin)` enters Π and awaits a decision. If logos accepts it, it enters F , and a more specific default `penguin  $\Rightarrow$   $\neg$ fly` appears in the theory. The operator o_{spec} selects it, and `fly` is cancelled.

Second order: first o_{spec} , then o_{abd} . The theory still has one default (`bird  $\Rightarrow$  fly`), there is no competitor, and `fly` is derived. Only after that does o_{abd} propose `is_a(tweety, penguin)`. Now, if the proposal is accepted, a second default appears in the theory, conflicting with the already drawn conclusion. This forces not a specificity selection but a *revision*: the previously derived consequence is reconsidered.

The same operators in a different order yield different theories: the process is historical, and each step changes the space of possible next steps.

The family does not reduce to a single operator. The expression $o_1 \cup o_2$ is not an operator: since the effect of o_2 depends on whether o_1 has already been applied, there is no function of the initial state Σ alone that would capture this pair.

External operators do not guarantee a result. o_{abd} may propose a contradictory hypothesis, o_{anal} an unsuitable mapping, o_{int} an erroneous answer. This is not a defect but the nature of doxa: it proposes, but does not guarantee. Internal operators, by contrast, are deterministic functions of F and their input; protocol operators guarantee determinism and verifiability, not truth.

We also note that composition of operators is not always defined: some pairs of operators are incompatible (for example, two operators each of which requires monopolistic access to Π). Idempotence is absent: repeated application of o_{abd} may generate a new hypothesis. Reversibility is also absent: cancelling a hypothesis changes the theory F , but the record of it remains in the journal $\mathcal{J}$. A full study of the algebraic properties of the family is the subject of a separate paper; here we fix only those properties that are essential for what follows.

3.8 The process of reasoning

Reasoning is a *sequence* of extensions:

$$\Sigma_0 \xrightarrow{o_1} \Sigma_1 \xrightarrow{o_2} \Sigma_2 \xrightarrow{o_3} \dots$$

where each step is an *act* of consulting doxa, marked by the operator that mediates it; fixation, classification, and application of each act are performed by the protocol. Not a “computation”, but a *consultation* of the source.

Each step is *recorded* in the *journal*: operator identifier, wave, rationale, source, and result (accepted / rejected / deferred). The journal is the *protocol* of the interaction between doxa and logos. It *is not* part of either doxa or logos. It is an *interface*, a protocol operator, not a function of doxa.

3.9 Limits of the model

The list of operators is *open*: the model does not claim that all have been identified. There may exist operators we do not see, and aspects of doxa that are not expressed through operators. This is not a temporary limitation but a consequence of the subject matter: doxa is not a theory in L , and its interaction with logos is described precisely by those functions that have been identified.

3.10 The role of logos

Logos is *not passive*. It *classifies* every proposal of doxa:

- **derivable** — the proposal already follows from F . Accepted as *narrative*.
- **cited** — the proposal is grounded in a *quotation* from the text. Added to the axioms.
- **hypothesis** — the proposal is new. Recorded in the journal with the mark H .
- **rejected** — the proposal fails verification. Discarded with a reason code.

Classification is a protocol function; the verdict it issues belongs to logos. Classification is *deterministic*: it depends on the *structure* of F and the *content* of the proposal, not on the “mood” of logos. This is the *guarantee* that logos does not “yield” to doxa. It *accepts* only what it *can* accept. And it *honestly* says when it *cannot*.

4 Implementation: Ankyra

In Section 3 we introduced the family of interaction operators between doxa and logos. Here we show that this model is not a theoretical construct: it is implemented in a working system, the hybrid neuro-symbolic reasoning system Ankyra [21]. We describe how each of the three roles is implemented — external operators, internal operators, and the protocol — and note that a full description of the architecture and its empirical evaluation is the subject of a separate paper [22].

4.1 Doxa is the LLM

What exactly the LLM brings into the system in the role of doxa. It is trained on texts, and text is not merely a means of transmitting information but a picture of the world: grammar, lexicon, and metaphors set the grid through which reality is reconstructed. The LLM is a hypertrophied version of that grid. It carries compressed human experience: the cultural field accessible through prediction of the next token; historicity frozen in probabilities. This is what makes it a bearer of doxa — not computability and not architecture, but the fact that it is a *cast* of human language and experience.

4.2 The principle: the LLM proposes, the engine decides

The core of Ankyra is the distinction between two participants. The LLM plays the role of doxa: it reads the text, decomposes it into structure, and in the reasoning cycle proposes additional steps. The symbolic engine plays the role of logos: it turns the structure into

a formal theory, closes it under inference, matches it against the question, and certifies what actually follows.

The boundary is absolute. The LLM never possesses truth. Nothing enters the theory that the engine cannot justify either by a verbatim quotation from the text or by an explicit hypothesis mark.

4.3 The proposal contract

Every statement the LLM wishes to add is deterministically classified into one of four categories: **derivable** (already follows from the theory), **cited** (supported by a verbatim quotation), **hypothesis** (knowledge not present in the text), **rejected** (fails verification). This is a direct implementation of the hypothesis generation operator (o_{abd}): the LLM proposes, the engine classifies, nothing is accepted without grounds. The distinction between **cited** and **hypothesis** makes the answers honest: a conclusion based on quotations is **proven**, a conclusion based on assumptions is **proven_under(H)** with a listing of those assumptions. An answer is never stronger than the grounds behind it.

4.4 The reasoning cycle

A task passes through four phases: *phase 0* — decomposition (the LLM decomposes the text into structure), *phase 1* — solving (the engine closes the theory), *phase 2* — the hint cycle (the LLM proposes one step at a time if the theory does not solve the question), *phase 3* — explanation (the chain is assembled mechanically from the proof history).

The hint cycle (phase 2) is organized into *waves*. A wave is one complete act of consulting doxa: the engine checks the current state of the theory and formulates the gaps, the LLM receives a hint with these gaps and proposes *exactly one* typed proposal, the engine classifies this proposal and applies or discards it, after which the wave is recorded in the journal and the cycle begins anew. The constraint “one proposal per wave” is principled: it makes each consultation of doxa verifiable and leaves a trace by which one can reconstruct at which step and why the answer changed.

The hint cycle implements the process of consulting doxa from Section 3: each wave is one act $F_i \xrightarrow{o} F_{i+1}$, mediated by an *external* operator, whereas fixation, classification, and application are performed by the protocol.

4.5 The journal

Each step is recorded in the journal: operator identifier, wave, rationale, source, result. The journal is the protocol of interaction between doxa and logos, not part of either. It implements the *protocol* role of the family (journal and fixation): it records each act of

consulting doxa and makes the process verifiable and reproducible. This is not a function of doxa.

4.6 Monotone base and moving answer

The base — axioms, rules, hypotheses — only grows: premises are never deleted, the goal is never weakened. But the answer may change as new information arrives. Each change is recorded as a **Revision** with an indication of the wave and the assumptions involved. This is an *internal* operator of the family (revision, o_{rev}): doxa supplies information, logos computes how it affects the theory, and the decision is recorded, not hidden.

4.7 Engines and the declared fragment

Ankyra is not a monolithic solver but an orchestrator of formalizations and decision procedures. Different kinds of tasks require different logics: Horn clauses (L0), stratified negation (L1), disjunction and quantifiers (L2), finite-domain constraints (L3), arithmetic (L4), defaults with specificity (D). These engines are *internal* operators of the family (the logotic side): L1 and D implement, respectively, cancellation (o_{def}) and specificity (o_{spec}), while L2 implements case analysis inside the decision procedure. None of them is a source of content.

The choice of procedure is carried out through the contract of the declared fragment: the engine examines the constructed theory and query and derives the set of required features. If the fragment fits within the available capabilities, the corresponding procedure is chosen; if not, the result is the named gap `out_of_fragment`, rather than a silent downgrade to a weaker logic. The declaration of the fragment is a *protocol* function; its application is internal, and the system honestly reports a mismatch rather than guessing.

4.8 The Arbitrator in the implementation

In Ankyra, the role of the Arbitrator is divided. The automatic Arbitrator is a *test runner*: it returns an objective verdict (the test passed or not) and does not depend on the proposals of doxa. The user Arbitrator is a *human* who accepts tasks, sets goals, and decides when the process is complete. The automatic Arbitrator is involved in phase 1 (when the goal is checked) and in phase 2 (when a proposal of doxa changes the theory); the user Arbitrator is involved in the external interface of the system. A full description of the Arbitrator’s role in the architecture is the subject of a separate paper [22].

4.9 Honesty by construction

The value of the system rests on guarantees provided mechanically: there are no facts in the theory belonging to the model; the question is never a source; there is no natural-language parsing in deterministic code; having assumed P , one cannot refute $\neg P$; an answer that used assumptions is always marked `proven_under(H)`. This is the implementation of the transparency property of the interface: logos accepts only what it can accept, and honestly says when it cannot.

4.10 Connection to the conceptual model

Ankyra implements the three roles of the family from Section 3. *External*: the proposal contract (hypothesis introduction operator, o_{abd}). *Internal*: revision (accounting of **Revision**), cancellation (L1), specificity (D). *Protocol*: journal, deterministic classification, grounding checks of quotations, wave protocol, and declaration of the fragment. The role of the *Arbitrator* is implemented as a composition of an automatic (test runner) and a user (human) component. Two external operators are not yet implemented: analogy (o_{anal}) and intuition (o_{int}). The system does not claim completeness: it implements what it can implement honestly, and honestly says what it cannot.

5 Discussion

We have introduced the distinction between doxa and logos, shown its formal analogues, and proposed the family of interaction operators between doxa and logos, implemented in Ankyra. Now we must discuss the limits of the approach: what it explains, what it does not explain, and what consequences follow from it.

5.1 What the model explains

The model shows how a formal system can expand. Gödel’s and Turing’s theorems describe the limits of closed systems. Our model shows that an open system — one that interacts with doxa through an explicit interface — can expand without that expansion being a derivation within it. The expansion comes from outside. Logos does not produce it. It accepts, rejects, or defers it.

Incompleteness is not eliminated but *transferred*: the resulting theory F' is again incomplete if it is consistent and sufficiently rich, but the point at which the system stops has shifted from closure to the interface. The system continues to work where a closed system would have stopped.

5.2 Limits of the approach

The model works with the functions of interaction, not with the content of doxa. This is a boundary of the subject matter, not a temporary limitation: doxa is not a theory in the language L , and there is nothing to describe in it — except those operations through which it extends a formal system. These operations are what the model is about.

The model also does not guarantee that the set of operators we have identified is complete. We regard it as an open list. The operators of analogy and intuition, not implemented in the current version, show that the space of operators is wider than what we have described. There may exist operators we do not see. This is not a deficiency of the model but a consequence of its subject matter.

Finally, the model does not provide a criterion for the “correct” choice among competing proposals of doxa. If two operators generate incompatible hypotheses, the system records the conflict but does not resolve it automatically. The choice remains with the *Arbitrator* (Section 3) — a human or an external program acting by a declared criterion. The model deliberately does not automate this step: no formal criterion resolves a dispute between incompatible proposals of doxa if both are compatible with the theory.

5.3 Connection to the oracle

In computation theory, the notion of an oracle was introduced by Turing to describe a machine that can obtain answers to questions not solvable by its own procedure [23]. The oracle is not part of the machine, but the machine can consult it. Doxa in our model plays a structurally close role.

There is, however, an important difference. The oracle gives *true* answers to questions of a certain class. Doxa does not give true answers: it gives *proposals* that may be true, false, or indeterminate. This is precisely why the interface to doxa includes classification — a proposal enters the system not as truth but as a hypothesis requiring verification. Logos does not trust doxa. It uses it.

5.4 Practical consequences

For explainable artificial intelligence, the model gives one clear principle: *an answer must not be stronger than its grounds*. If the system derived a conclusion from quotations — that is one thing. If from assumptions — another. If from a mixture — a third, and this third must be explicitly marked. The user has the right to know what the answer rests on.

For system verification, this means that verifiability does not require full formalization. It is enough that each step is traceable to a source and that each extension is recorded in

the journal. Doxa remains non-formalizable, but its *trace* in the system is formal.

For the architecture of hybrid systems, this means that the division between the neural and symbolic parts is not an engineering compromise. It is a structural division reflecting the distinction between doxa and logos. The neural part is not “weaker” than the symbolic. It does what the symbolic cannot do. The symbolic part is not “stricter” than the neural. It does what the neural must not do. Each side is strong in its own way.

5.5 What remains open

The model leaves several questions open. How do operators of different roles relate to one another — are there laws linking external, internal, and protocol steps, identities, priorities? How does one measure the effectiveness of an operator — by the depth of change in the theory, by the number of accepted proposals, by the contribution to the final answer? How does one distinguish an operator that genuinely extends the system from one that merely reformulates what is already known? These questions have no answer within the present work. They point to the direction of further research.

6 Related work

6.1 Map of the field

A 2026 systematic survey [24] (178 works, 2020–2025) identifies four stable patterns of neuro-symbolic integration: sequential cascade, symbolic validator, symbolic planner with a neural executor, and differentiable integration. Ankyra is closest to the sequential cascade with a bounded cycle, but it differs in two respects: a proposal protocol under which every statement enters the theory only with a quotation or an assumption tag, and a declared fragment under which a mismatch between the required and available logic yields an honest `out_of_fragment` rather than a silent downgrade to a weaker procedure. The same survey notes that meta-cognitive abilities — monitoring one’s own conclusions, recognizing a lack of knowledge, switching modes — appear in only 5% of the works; `out_of_fragment` and `proven_under(H)` can be seen as a contribution to exactly this niche.

The closest system in spirit is *Autonomous Trustworthy Agents* (ATA) [25]: it also pursues determinism and checkability, but achieves them by *removing* the LLM from the runtime entirely (the LLM builds a knowledge base offline; only the symbolic solver runs online). Ankyra keeps the LLM in the loop, constraining it through the protocol; the two are different points on the same spectrum. The same approach to verifying regulatory documents with an SMT solver is described in [26].

6.2 Language model and symbolic solver

Translator-plus-solver pipelines (Logic-LM [27], LINC [28]) also hand reasoning over to a symbolic solver, but trust the translation entirely: if the formalization is wrong, so is the conclusion. In our model the boundary is stricter: every statement enters the theory only with a verbatim quotation or an explicit assumption tag, answer strength is derived from the premises (`proven` versus `proven_under`), and a mismatch between the required and available logic yields an honest `out_of_fragment` rather than a hidden downgrade to a weaker procedure. An empirical comparison of Ankyra with Logic-LM and LINC on common input is given in the companion paper [22].

6.3 Non-monotonic reasoning and defaults

Default logic [9] and circumscription [10] formalize the defeasibility of conclusions; Ankyra implements a defaults layer with specificity via `is_a` and honestly reports an undecided conflict rather than guessing a winner. The philosophical framing (doxa/logos) introduced in this paper differs from purely engineering approaches to separating the LLM and the solver: it derives an architectural principle from an epistemological distinction going back to the ancient tradition.

6.4 Proof history and checkability

The reliance on an automatically assembled proof history aligns Ankyra with work on checkable proof. The explanation here is a presentation of the derivation, not a second opinion about it: the model may not add facts to it, and every step names the rule that fired, the bindings used, and the supporting quotation or hypothesis.

7 Conclusion

We have proposed an interpretation of the incompleteness theorems not as absolute limits of cognition but as limits of static systems. Human thinking is not a static system: it develops by relying on doxa — a non-formalizable source of lived experience, the cultural field, and bodily knowledge. The distinction between doxa and logos has formal analogues in modern logic, but none of them formalizes doxa as such.

Our contribution consists in the family of interaction operators between doxa and logos — a formal interface between a source of proposals and a formal system — and in the fact that this interface is *asymmetric*: external operators supply content, internal ones decide, and protocol ones make the exchange verifiable. We have implemented this model in the Ankyra system, where a large language model plays the role of doxa and a symbolic solver plays the role of logos [21]. The principle “the LLM proposes, the engine decides”

ensures transparency: nothing enters the theory without grounds, and an answer is never stronger than the grounds behind it.

We do not claim to have solved the problem of incompleteness. We claim to have shown how a system can work with this problem without attempting to overcome it. Doxa does not cancel logos — it nourishes it. And in this interaction there is not a defeat of formalization but its extension.

References

- [1] Gödel K. Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I // Monatshefte für Mathematik und Physik. 1931. Bd. 38. S. 173–198.
- [2] Turing A. M. On Computable Numbers, with an Application to the Entscheidungsproblem // Proceedings of the London Mathematical Society. 1936. Ser. 2, Vol. 42. P. 230–265.
- [3] Diels H., Kranz W. Die Fragmente der Vorsokratiker. Berlin: Weidmann, 1903–1910.
- [4] Plato. Theaetetus // Plato. Collected Works: in 4 vols. Moscow: Mysl, 1990–1994. Vol. 2.
- [5] Aristotle. Nicomachean Ethics // Aristotle. Collected Works: in 4 vols. Moscow: Mysl, 1975–1983. Vol. 4.
- [6] Hintikka J. Knowledge and Belief: An Introduction to the Logic of the Two Notions. Ithaca: Cornell University Press, 1962.
- [7] Segerberg K. The Basic Dynamic Doxastic Logic of AGM // Frontiers in Belief Revision / Ed. by M.-A. Williams, H. Rott. Dordrecht: Kluwer, 2001. P. 57–84.
- [8] Segerberg K. Some Completeness Theorems in the Dynamic Doxastic Logic of Iterated Belief Revision // The Review of Symbolic Logic. 2010. Vol. 3, No. 1. P. 1–24.
- [9] Reiter R. A Logic for Default Reasoning // Artificial Intelligence. 1980. Vol. 13, No. 1–2. P. 81–132.
- [10] McCarthy J. Circumscription—A Form of Non-Monotonic Reasoning // Artificial Intelligence. 1980. Vol. 13, No. 1–2. P. 27–39.
- [11] Moore R. C. Semantical Considerations on Nonmonotonic Logic // Artificial Intelligence. 1985. Vol. 25, No. 1. P. 75–94.
- [12] Peirce C. S. The Beginnings of Pragmatism. St. Petersburg: Aletheia, 2000.

- [13] Peirce C. S. Lectures on Pragmatism (1903) // The Collected Papers of Charles Sanders Peirce. Cambridge: Harvard University Press, 1931–1958. Vol. 5.
- [14] Niiniluoto I. Defending Abduction // Philosophy of Science. 1999. Vol. 66, No. 3. P. S436–S451.
- [15] Kakas A. C., Kowalski R. A., Toni F. Abductive Logic Programming // Journal of Logic and Computation. 1992. Vol. 2, No. 6. P. 719–770.
- [16] Vinkov M. M., Fominykh I. B. Temporal Non-Monotonic Logical Systems: Interrelations and Computational Complexity // Artificial Intelligence and Decision Making. 2008. No. 4. P. 19–25.
- [17] Cadoli M., Schaerf M. A Survey of Complexity Results for Non-Monotonic Logics // Journal of Logic Programming. 1993. Vol. 17, No. 2–4. P. 127–160.
- [18] Eiter T., Gottlob G. Propositional Abduction is Σ_2^P -Complete // Information Processing Letters. 1995. Vol. 55, No. 4. P. 213–218.
- [19] Eiter T., Gottlob G. The Complexity of Logic-Based Abduction // Journal of the ACM. 1995. Vol. 42, No. 1. P. 3–42.
- [20] Alchourrón C. E., Gärdenfors P., Makinson D. On the Logic of Theory Change: Partial Meet Contraction and Revision Functions // The Journal of Symbolic Logic. 1985. Vol. 50, No. 2. P. 510–530.
- [21] Ankyra: A domain-agnostic neuro-symbolic reasoning engine. GitHub repository. URL: <https://github.com/merfill/ankyra>
- [22] Lapshin V. Ankyra: Sound Neuro-Symbolic Reasoning with Verifiable Proofs. (companion paper, in preparation).
- [23] Turing A. M. Systems of Logic Based on Ordinals // Proceedings of the London Mathematical Society. 1939. Ser. 2, Vol. 45. P. 161–228.
- [24] Bin Hakim S., Adil M., Velasquez A., Song H. H. Neuro-symbolic agentic AI: Architectures, integration patterns, applications, open challenges and future research directions // Computer Science Review. 2026. Vol. 60. Art. 100902. DOI: 10.1016/j.cosrev.2026.100902.
- [25] Peer D., Stabinger S. ATA: Autonomous Trustworthy Agents. arXiv:2510.16381.
- [26] Hsia Y.-S., Yu F., Jiang J.-H. R. Neuro-Symbolic Compliance: LLM-driven SMT Verification of Regulatory Documents. arXiv:2601.06181.
- [27] Pan L. et al. Logic-LM: Empowering Large Language Models with Symbolic Solvers for Faithful Logical Reasoning. Findings of EMNLP. 2023.

- [28] Olausson T. X. et al. LINC: A Neurosymbolic Approach for Logical Reasoning by Combining Language Models with First-Order Logic Provers. EMNLP. 2023.
- [29] The JSM Method for Automatic Hypothesis Generation: Logical and Epistemological Foundations / comp. by O. M. Anshakov, E. F. Fabrikantova; ed. by O. M. Anshakov. Moscow: URSS, 2009. 430 p. (in Russian).
- [30] Finn V. K. Epistemological Foundations of the JSM Method for Automatic Hypothesis Generation // Voprosy Filosofii. 2014. No. 2. P. 83–96. (in Russian).
- [31] Finn V. K. Epistemological foundations of the JSM method for automatic hypothesis generation // Automatic Documentation and Mathematical Linguistics. 2014. Vol. 48, No. 2. P. 96–148.